《第7组 文件存储讲解》10 分钟录屏脚本（指读版）

> 交付要求：视频命名 `第7组 文件存储讲解`，今晚 20:00 前交。
> 结构：**界面演示约 4 分钟 + 程序讲解约 5 分钟**（老师要求的两部分），覆盖老师 5 问。
> 加粗句子 = 照念的"标准答案"，其余是操作提示。每段标了目标时长，念完一段看一次表。
>
> **【代码讲解不用看懂】** 第三节每一步都给了：打开哪个文件 → 按 Ctrl+F 搜哪个英文词 → 鼠标停在哪。搜索词就像"坐标"，搜到后把那几行放大（Ctrl + 鼠标滚轮），手指着屏幕，照加粗句子念，嘴和代码就对上了。念之前先看一眼【人话】两行，知道自己在说什么就行。

〇、录屏前准备清单（开录前 5 分钟做完）

1. 启动 MySQL（3306）、Redis（6379）
2. 起后端：`backend` 目录 `mvn spring-boot:run`，看到 `Tomcat started on port 8080`
3. 起前端：`frontend` 目录 `npm run dev`，打开 `http://localhost:5173/files`
4. （可选，想要干净演示）清空历史：`mysql -uroot -p -D scenic_tour -e "DELETE FROM sys_file;"`，删除 `backend/uploads` 目录
5. 点页面「生成 20MB 演示文件」按钮 → 文件进下载目录
6. 浏览器缩放 100%，**先按 `Win + G` → 圆点开始录制**（或 `Win + Alt + R`），确认小黄条出现
7. （讲解环节用）IDEA 打开项目，按两下 `Shift` 分别搜 `FileStorageService`、`SysFileMapper`、`FileView.vue` 三个文件并打开成标签页，录屏时直接点标签切换

一、开场（0:00 – 0:30）

「大家好，我是第 7 组的 XXX。下面用不到 10 分钟讲解我们组文件存储模块的上传和下载实

现：先做界面演示，再对着代码讲清楚分片重组、坏块重传、批量上传、断点续传、进度条和高并发下载六个问题。文件元数据存在 MySQL 的 sys_file 表，文件内容存在服务器磁盘，前端是 Vue3，后端是 Spring Boot。」

操作：页面停在 `/files`，三个卡片 + 底部日志区可见。

二、界面演示（0:30 – 4:30）

2.1 大文件分片上传 + 断点续传（约 1 分 40 秒）

操作顺序：

1. 卡片①选「演示大文件-20mb.pdf」→ 分片大小选 **256 KB（演示用小分片）**
2. 点「开始上传」→ 指着日志念：
   「前端先用 spark-md5 分块增量算出整个文件的 md5，再调 init 接口。后端按 md5 判断三件事：服务器有相同 md5 的完成文件就走秒传；有传了一半的就接着传；都没有就新建分片任务。这次是新建，20MB 按 256KB 切成 79 片，3 路并发上传。」
3. 进度条走到 30%~40% 时点「暂停」，指日志：
   「现在模拟浏览器关闭或断网，请求全部中止。」
4. 等 2 秒点「继续（断点续传）」，指日志：
   「继续时前端调 progress 接口，拿到服务器已收到的分片号列表，只补缺失的片，已经传过的片一个字节都不会重传——这就是上传的断点续传。」
5. 等传完，指日志最后一条：
   「所有分片到齐后调 complete，后端核对没有缺片，按分片号顺序合并，再算整体 md5 和 init 时声明的比对，通过后落库，返回下载地址。」

2.2 模拟分块损坏（约 1 分钟）

1. **先点「生成 20MB 演示文件」重新生成一个新文件**（每次生成的内容都是随机字节，md5 必不同）——不要用刚传过的旧文件，否则 init 直接秒传跳过，到不了坏块环节
2. 勾选「模拟分片损坏（篡改第 2 片 1 个字节）」→ 选这个新文件 → 开始上传
3. 日志出现「已人为篡改第 2 片」和「后端校验：第 2 片 md5 不一致 → 自动重传原片」时暂停滚动：
   「每个分片上传时请求头带这一片内容的 md5，后端落盘前重新算一遍，不一致就返回错

误并指明第几片坏了——前端收到后自动重传这一片，这就是分块损坏后的通知重传机制。」

4. 等上传完成。

2.3 小文件批量上传（约 40 秒）

1. 卡片②点选择框，按住 Ctrl 多选 3 个小文件（可以现用记事本存 3 个 .txt/.md）
2. 点「批量上传」，三条日志变 ✓：
   「5MB 以内的小文件走 simple 直传接口。批量上传的做法是：接口保持单个文件的简单职责，前端多选之后并发循环调用——三次请求同时发出去，后端不需要专门的批量接口。」

2.4 下载进度条 + 下载断点续传（约 50 秒）

1. 卡片③文件列表点「下载」，进度条到 40%~50% 点「暂停」，再点「继续（Range 续传）」：
   「下载用 fetch 流式读取，每收到一段就累计字节，除以响应头里的 Content-Length 就是进度。暂停后浏览器记下已收到的字节数，继续时请求头带 Range: bytes=已收字节数-，服务器返回 206 Partial Content，从断点接着传——这就是下载的断点续传。」
2. 文件保存成功，日志显示「下载完成」。

2.5 秒传彩蛋（约 20 秒）

「最后看秒传：同一个文件再传一次，init 一比对 md5，服务器已经有了，前端一个字节都不用发，直接拿到下载地址。」

三、程序讲解（4:30 – 9:30）—— 指读版

> 每步三行：**【操作】**怎么定位代码（照做就行）**、【人话】**这几行代码在干嘛（念之前扫一眼，心里有底）**、【照念】**对着屏幕说的话。
> 字体太小就 `Ctrl + 鼠标滚轮` 放大。

问题 1：大文件如何分片、后台如何重组、坏块如何通知重传（约 1 分 40 秒）

第 1 停：坏块校验

- 【操作】IDEA 标签切到 `FileStorageService` → `Ctrl+F` 搜 `md5DigestAsHex` → 高亮它下面那行 `if (claimedChunkMd5 != null ...)` 到 `throw new BizException(...)`（一共 3 行）
- 【人话】前端每传一片，请求头里带了"这一片应该是什么 md5"；后端拿到内容自己算一遍真实 md5，两个对不上 = 传输途中损坏了 → 抛错并告诉前端"第几片坏了"
- 【照念】
  「分片在前端做：按 256KB 或 5MB 用 Blob.slice 切，每一片算好自己的 md5 放进请求头 X-Chunk-Md5。后端 saveChunk 收到片，用 Spring 的 DigestUtils 重新计算内容 md5，跟请求头比对——这是第一层校验。不一致就抛 BizException，错误码 10001，消息里带分片号，data 里也带分片号，前端拿到就自动重传这一片。」

第 2 停：原子登记（切文件）

- 【操作】标签切到 `SysFileMapper` → 搜 `JSON_ARRAY_APPEND` → 高亮整条 SQL（UPDATE ... 到 AND JSON_CONTAINS 那行）
- 【人话】每收好一片，就往数据库的"已收分片名单"（uploaded_chunks 数组）里追加这个片号；JSON_CONTAINS 保证名单里没有才追加，同一片传两次也不会记两次
- 【照念】
  「校验通过先落盘成 1.part、2.part，再用这条 SQL 原子登记：JSON_ARRAY_APPEND 往 uploaded_chunks 数组追加，WHERE 里 JSON_CONTAINS 保证同一片不会登记两次，并发下也安全。」

第 3 停：后台重组

- 【操作】标签切回 `FileStorageService` → 搜 `public String complete` → 先高亮 `List<Integer> missing = IntStream...` 那 2 行，念到"第二步"时往下滚到 `for (int no = 1; no <= file.getChunkTotal(); no++)` 高亮循环体，念到"第三步"时搜 `MERGE_MD5_MISMATCH` 高亮那个 if
- 【人话】①缺片核对：1 到总片数，挨个看数据库名单里有没有，缺的把片号列表返回给前端；②合并：按片号顺序把每个 .part 文件拼接写出，边拼边算整体 md5；③终校：拼出来的整体 md5 跟 init 时声明的对不上 = 合并出问题 → 删脏文件、任务标记失败、返回 50003
- 【照念】

「后台重组分四步：第一步完整性核对，拿数据库里的 uploaded_chunks 和 1 到 chunkTotal 比对，缺哪些片就把缺失清单放进 data 返回，前端只补缺失的——这是第二层坏块通知；第二步用 FileChannel 按分片号顺序把 .part 文件写进最终目录；第三步整体 md5 终校，合并时边写边算，结果跟 init 声明的 md5 不一致就说明合并出了问题，删掉脏文件、把任务标记失败、返回 50003 让用户重新上传；第四步更新状态为完成。三层校验保证：单片坏、丢片、合并错，都能发现并自动恢复。」

问题 2：小文件如何批量上传（约 30 秒）

第 4 停：后端单接口

- 【操作】还在 `FileStorageService` → 搜 `SIMPLE_MAX_SIZE` → 高亮 `if (ALLOWED_TYPES.contains...)` 和 `if (file.getSize() > SIMPLE_MAX_SIZE...)` 两行
- 【人话】小文件直传只有两道门：类型白名单、大小不超 5MB
- 【照念】
  「小文件单接口 simpleUpload：先过类型白名单 pdf、txt、md、docx，再校验不超过 5MB，超了就提示走分片通道。批量上传不在后端做——」

第 5 停：前端并发循环（切文件）

- 【操作】标签切到 `FileView.vue` → 搜 `uploadSmall` → 高亮 `await Promise.all(smallFiles.value.map(...)` 那行
- 【人话】前端把多选的几个文件，同时（并发）一个个调上面那个单文件接口
- 【照念（接着上一句）】
  「前端 FileView 里多选文件后用 Promise.all 并发循环调用 simple 接口。好处是后端接口保持简单可复用，批量逻辑完全由前端编排。」

问题 3：断点续传（约 1 分 20 秒）

第 6 停：上传侧靠数据库名单

- 【操作】标签切到 `SysFileMapper` → 搜 `JSON_CONTAINS`（第 1 停那条 SQL 还在）→ 高亮 `WHERE id = #{id} AND status = 0 AND JSON_CONTAINS...`；再切 `FileStorageService` 搜 `断点续传`，高亮 `partial != null` 那个 if 块
- 【人话】服务器给每个没传完的任务记着"已收分片名单"，客户端重新来，查名单发剩下的
- 【照念】
  「sys_file 表的 uploaded_chunks 是 JSON 数组，记录服务器已收到的分片号。任务中

断后重新 init，后端查到同 md5 的未完成任务，把这个数组原样返回；前端从 1 循环到 chunkTotal，数组里已有的直接跳过——所以不管中断多少次，永远只传缺的那部分。」

第 7 停：下载侧靠 Range

- 【操作】`FileStorageService` 搜 `skipFully` → 高亮整个 `StreamingResponseBody body = out -> { ... }` 块（约 6 行）；再往上滚一点高亮 `if (rangeHeader != null && rangeHeader.matches(...))` 和 `status = 206`
- 【人话】浏览器说"我要从第 5 个字节开始要"（Range 头），服务器就：状态码回 206、跳过前 5 个字节、从断点开始流式吐文件
- 【照念】
  「下载侧断点续传用 HTTP 标准的 Range：后端用正则解析 bytes=start-end，越界返回 400；合法就把状态码改成 206，响应头加 Content-Range: bytes 起止/总长，再用 StreamingResponseBody，skipFully 跳过前面已传的字节，从断点位置开始流式输出。浏览器收到 206 就从断点继续拼文件。」

问题 4：进度条（约 50 秒）

第 8 停：上传进度 = 分片进度 + 当前片实时进度

- 【操作】标签切到 `FileView.vue` → 搜 `onUploadProgress` → 高亮那 3 行；再搜 `refreshProgress` → 高亮 `uploadProgress.value = Math.min(99, ...)` 那行
- 【人话】每片上传时 axios 报告"这片传了多少字节"，加上已经传完的片的字节，除以总大小 = 总进度
- 【照念】
  「上传进度条是加权算出来的：已完成分片的字节数是确定的部分，加上当前正在传的每一片通过 axios 的 onUploadProgress 拿到实时 loaded，两者相加除以文件总大小。所以进度条既反映分片粒度也反映片内粒度，不会出现长时间卡住不动。」

第 9 停：下载进度 = 已收字节 / 总字节

- 【操作】还搜 `getReader` → 高亮 `t.received += value.length` 和 `dlState.progress = Math.round(...)` 两行
- 【人话】下载每收到一块就累加，除以总长就是百分比
- 【照念】
  「下载进度条更直接：fetch 的 reader 每读一段累加 received，除以 Content-Length，就是精确到字节的百分比。」

问题 5：如何支持高并发下载（约 40 秒）

第 10 停：无锁流式输出

- 【操作】标签切回 `FileStorageService` → 搜 `newInputStream` → 高亮 try 那几行（64KB 缓冲循环 copy 在 `copy(in, out, fLength)` 里）
- 【人话】每个下载请求都是独立的"边读磁盘边发网络"，服务器内存里只过手 64KB 小缓冲，不存整个文件，所以并发再多内存也不涨
- 【照念】
  「这个下载接口天生扛并发，三个原因：第一，它无锁无状态——不查全局锁、不在内存里攒整个文件，每个请求一个独立的 StreamingResponseBody，边读磁盘边写网络，内存占用是固定的 64KB 缓冲区，来一千个请求也就一千个小缓冲；第二，Range 支持让多线程分段下载成为可能，下载器可以把一个文件拆成几段同时拉；第三，生产环境前面加 Nginx，命中静态规则直接 sendfile 零拷贝，根本不进 Java 进程。对比错误做法——把文件整个读进内存再返回——一台机器几十个并发就把堆打爆了。」

四、收尾（9:30 – 10:00）

「总结一下：文件内容放磁盘、元数据放 MySQL；大文件分片上传、三层校验（单片 md5、缺片清单、整体终校）；小文件前端并发批量直传；上传下载都支持断点续传；进度条前后端配合加权计算；下载接口无锁流式加 Range，天然支持高并发。第 7 组文件存储讲解到此结束，谢谢老师。」

操作：`Win + G` → 停止录制 → 视频改名 `第7组 文件存储讲解` → 交课代表。

附 1：万一现场翻车的兜底

情况	话术
秒传提前触发（重复演示同一文件）	顺势讲：「正好演示秒传——md5 相同直接返回地址，0 字节传输」
MySQL 没起 / 连接失败	50000 说明在后端 GlobalExceptionHandler

情况	话术
	兜底，重启 MySQL 后再录
分片太快看不清日志	演示时用 256KB 分片 + 20MB 文件，79 片足够看清过程
暂停后点继续没反应	继续按钮依赖 progress 接口，确认后端 8080 还活着；重新选同一文件点开始上传也能续传（init 断点续传分支）
Ctrl+F 搜不到英文词	确认 IDEA 左下角语言无关化：搜的词直接从本表复制粘贴进去

附 2：10 个搜索词速查卡（贴在手边，讲到哪搜到哪）

#	搜什么	在哪个文件	对应问题
1	`md5DigestAsHex`	FileStorageService	1 坏块校验
2	`JSON_ARRAY_APPEND`	SysFileMapper	1 登记分片
3	`public String complete`	FileStorageService	1 重组
4	`SIMPLE_MAX_SIZE`	FileStorageService	2 小文件
5	`uploadSmall`	FileView.vue	2 批量
6	`断点续传`	FileStorageService	3 上传续传
7	`skipFully`	FileStorageService	3 Range/5 流式
8	`onUploadProgress`	FileView.vue	4 上传进度
9	`getReader`	FileView.vue	4 下载进度
10	`newInputStream`	FileStorageService	5 高并发